

MSCI222 Coursework

Prepared by:

38557568

38802139

Introduction

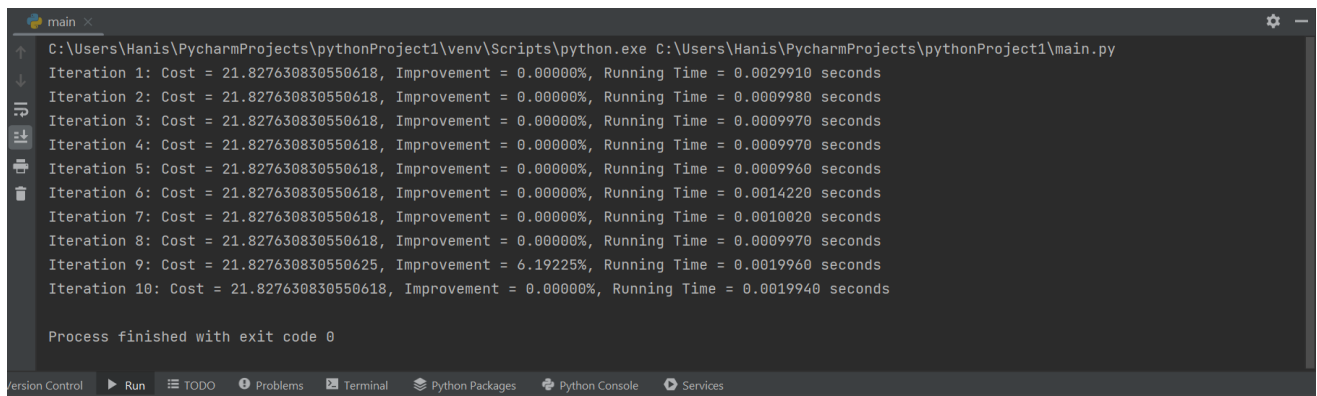
In this report, we analyse and interpret our findings and attempts to solve the TSP problem. We used the heuristics method learned in lectures and workshop sessions to solve TSP. We used and modified the codes algorithms learned in workshop to better assess and improve the performance of the algorithms. We focused on the use of constructive algorithms as a start point to create initial solutions, then utilise the local search algorithms for iterative improvement. With several experimentations, we tested our solution code on different instances provided on Moodle and shifted our focus to the cost and improvement over iterations. Hence, with the heuristic approach, this report aims to clearly assess the performance and applicability of these heuristic approaches in solving TSP problems. Our codes of algorithm for the questions are attached in the appendix section of the report.

Evaluating the performance of algorithms codes

Question 1

To tackle this question, we used the nearest neighbour algorithm to create an initial solution for the problem, picking a starting point basically based on the nearest cities. We had used our knowledge and the codes learned from workshop sessions. Next, we used the steepest descent method to alter the initial solution by making small changes. We had run our code for this question for three times on different instances and analysed it below. The outputs clearly show that our algorithm performs well initially and managed to significantly improve the solution at iteration 9. This proves our algorithm made a good one-time improvement.

First experimentation (on Ex5.txt)



```
main x
C:\Users\Hanis\PycharmProjects\pythonProject1\venv\Scripts\python.exe C:\Users\Hanis\PycharmProjects\pythonProject1\main.py
Iteration 1: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0029910 seconds
Iteration 2: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0009980 seconds
Iteration 3: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0009970 seconds
Iteration 4: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0009970 seconds
Iteration 5: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0009960 seconds
Iteration 6: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0014220 seconds
Iteration 7: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0010020 seconds
Iteration 8: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0009970 seconds
Iteration 9: Cost = 21.827630830550625, Improvement = 6.19225%, Running Time = 0.0019960 seconds
Iteration 10: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0019940 seconds

Process finished with exit code 0
```

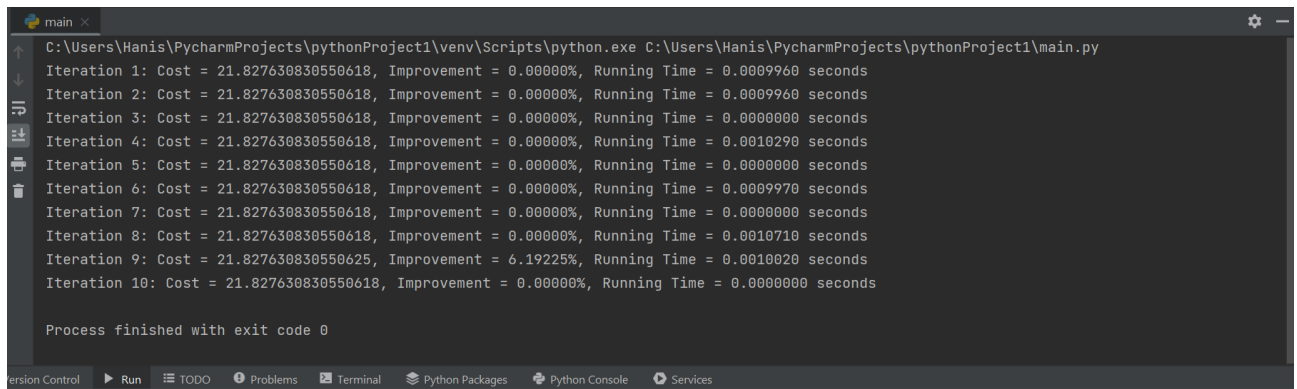
Above diagram is the output from Pycharm for the code in Question 1. The cost of the solution and the improvement over each iteration is shown.

Firstly, the cost means the total distance of the TSP route after each iteration. The value is 21.827630830550618. Throughout iterations 1 until 8 and iteration 10, the algorithm found it's best route in these iterations which is the constant value stated. On the other hand, the running time vary slightly but remain low which is good.

Next, it can be seen that there is an improvement of 6.19225% in iteration 9. This suggests that the algorithm found a better route than before. Hence, to further analyse whether the improvement in iteration 9 was just an irregularity or instead consistent, we decided to run the algorithm for 2 more times on different instances provided in Moodle to further understand.

Below are the diagrams of the 2 other results of attempts of running the code again on 2 other instances file.

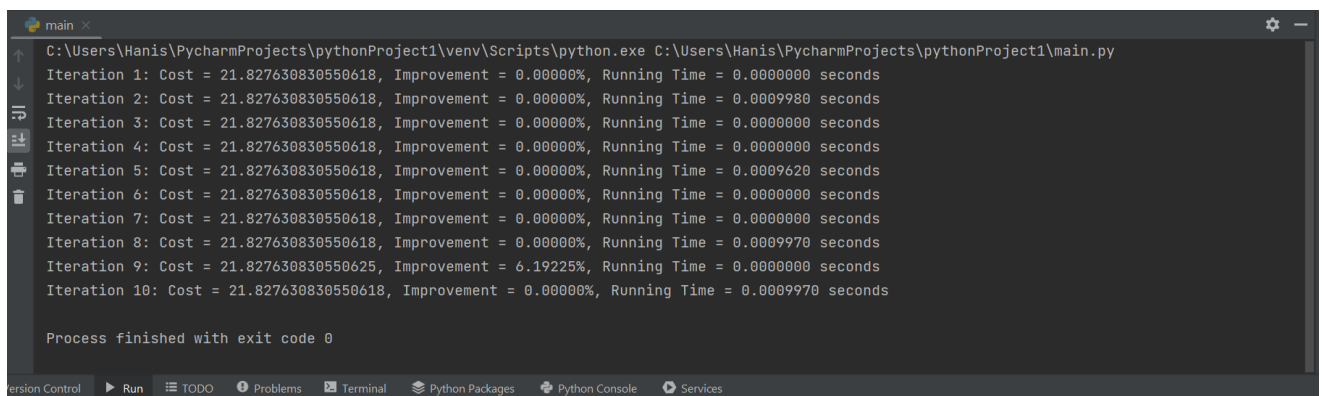
Second Experimentation (on Uly16.txt)



```
main x
C:\Users\Hanis\PycharmProjects\pythonProject1\venv\Scripts\python.exe C:\Users\Hanis\PycharmProjects\pythonProject1\main.py
Iteration 1: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0009960 seconds
Iteration 2: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0009960 seconds
Iteration 3: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0000000 seconds
Iteration 4: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0010290 seconds
Iteration 5: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0000000 seconds
Iteration 6: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0009970 seconds
Iteration 7: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0000000 seconds
Iteration 8: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0010710 seconds
Iteration 9: Cost = 21.827630830550625, Improvement = 6.19225%, Running Time = 0.0010020 seconds
Iteration 10: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0000000 seconds

Process finished with exit code 0
```

Third Experimentation (on u159.txt)



```
main x
C:\Users\Hanis\PycharmProjects\pythonProject1\venv\Scripts\python.exe C:\Users\Hanis\PycharmProjects\pythonProject1\main.py
Iteration 1: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0000000 seconds
Iteration 2: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0009980 seconds
Iteration 3: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0000000 seconds
Iteration 4: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0000000 seconds
Iteration 5: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0009620 seconds
Iteration 6: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0000000 seconds
Iteration 7: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0000000 seconds
Iteration 8: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0009970 seconds
Iteration 9: Cost = 21.827630830550625, Improvement = 6.19225%, Running Time = 0.0000000 seconds
Iteration 10: Cost = 21.827630830550618, Improvement = 0.00000%, Running Time = 0.0009970 seconds

Process finished with exit code 0
```

Running the code for the second and third time resulted in the above outputs in shown diagram. It can be observed that the cost in the first eight iterations remain constant which is 21.827630830550618, with iteration 10 showing no improvement every single time, and iteration 9 constantly shows a significant improvement of 6.19225% in all three runs.

From the major improvement of 6.19225% in iteration 9, it indicates a systematic behaviour of the algorithm rather than an anomaly. However, the performance is consistent, allows for good dependency. The consistency from all three runs of our code on different instances proves our method is reliable.

Question 2

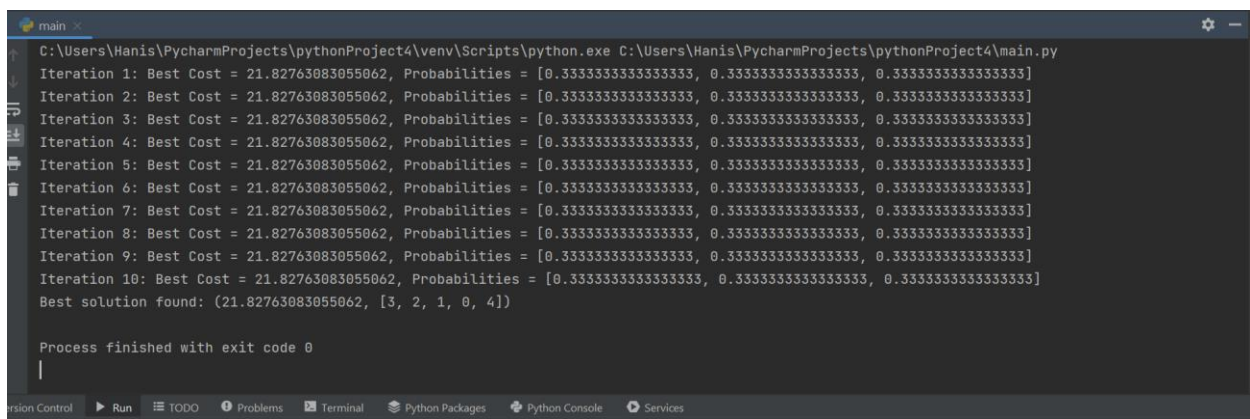
For question 2, we developed a code that implements one constructive algorithm and two local search algorithms: the 'nearest_neighbour_algorithm' to construct an initial solution; and the 'tsp_local_search_1' (2-opt method) and 'tsp_local_search_2' (city swap) algorithms to search and provide an optimised, lowest cost solution.

Question 3

The *nearest_neighbour_algorithm* is a constructive algorithm used to generate an initial solution for the TSP. This algorithm starts from a random city and always moves to the nearest unvisited city, creating a loop that visits each city exactly once. This initial solution is then put through search algorithms (*local_search1*, *local_search2*, *local_search3*), each of which have a different approach (2-opt, swap cities, simple city move) and through a variable search network that chooses which search algorithm to iterate through based on probability. Each search algorithm begins with an equal percentage of being chosen, and better performing algorithms are given a higher probability of being chosen. this optimises the process of finding a solution by allowing better-performing algorithms to be used more frequently.

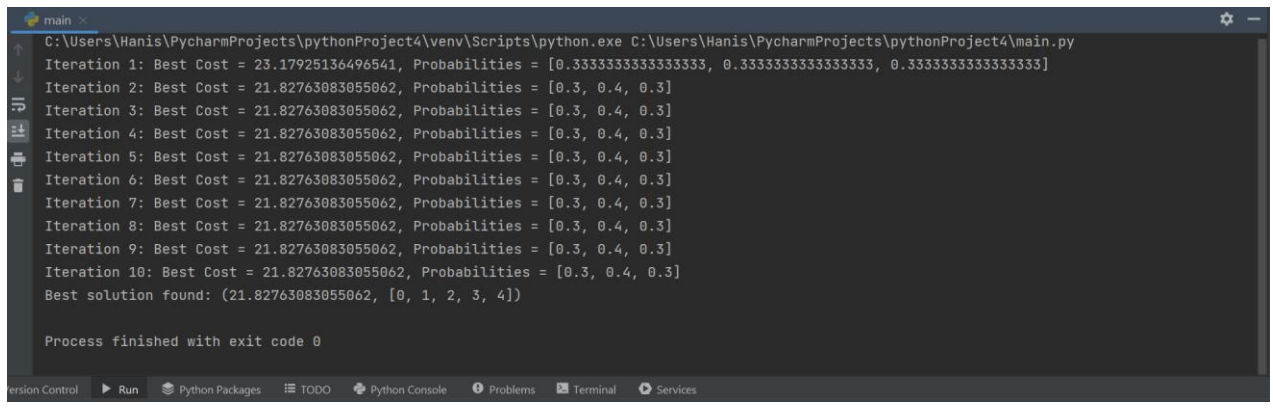
We developed our code and did 3 experimentations on it.

First experimentation (on Ex5.txt)



```
main
C:\Users\Hanis\PycharmProjects\pythonProject4\venv\Scripts\python.exe C:\Users\Hanis\PycharmProjects\pythonProject4\main.py
Iteration 1: Best Cost = 21.82763083055062, Probabilities = [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
Iteration 2: Best Cost = 21.82763083055062, Probabilities = [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
Iteration 3: Best Cost = 21.82763083055062, Probabilities = [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
Iteration 4: Best Cost = 21.82763083055062, Probabilities = [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
Iteration 5: Best Cost = 21.82763083055062, Probabilities = [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
Iteration 6: Best Cost = 21.82763083055062, Probabilities = [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
Iteration 7: Best Cost = 21.82763083055062, Probabilities = [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
Iteration 8: Best Cost = 21.82763083055062, Probabilities = [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
Iteration 9: Best Cost = 21.82763083055062, Probabilities = [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
Iteration 10: Best Cost = 21.82763083055062, Probabilities = [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
Best solution found: (21.82763083055062, [3, 2, 1, 0, 4])

Process finished with exit code 0
```



```
main
C:\Users\Hanis\PycharmProjects\pythonProject4\venv\Scripts\python.exe C:\Users\Hanis\PycharmProjects\pythonProject4\main.py
Iteration 1: Best Cost = 23.17925136496541, Probabilities = [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
Iteration 2: Best Cost = 21.82763083055062, Probabilities = [0.3, 0.4, 0.3]
Iteration 3: Best Cost = 21.82763083055062, Probabilities = [0.3, 0.4, 0.3]
Iteration 4: Best Cost = 21.82763083055062, Probabilities = [0.3, 0.4, 0.3]
Iteration 5: Best Cost = 21.82763083055062, Probabilities = [0.3, 0.4, 0.3]
Iteration 6: Best Cost = 21.82763083055062, Probabilities = [0.3, 0.4, 0.3]
Iteration 7: Best Cost = 21.82763083055062, Probabilities = [0.3, 0.4, 0.3]
Iteration 8: Best Cost = 21.82763083055062, Probabilities = [0.3, 0.4, 0.3]
Iteration 9: Best Cost = 21.82763083055062, Probabilities = [0.3, 0.4, 0.3]
Iteration 10: Best Cost = 21.82763083055062, Probabilities = [0.3, 0.4, 0.3]
Best solution found: (21.82763083055062, [0, 1, 2, 3, 4])

Process finished with exit code 0
```

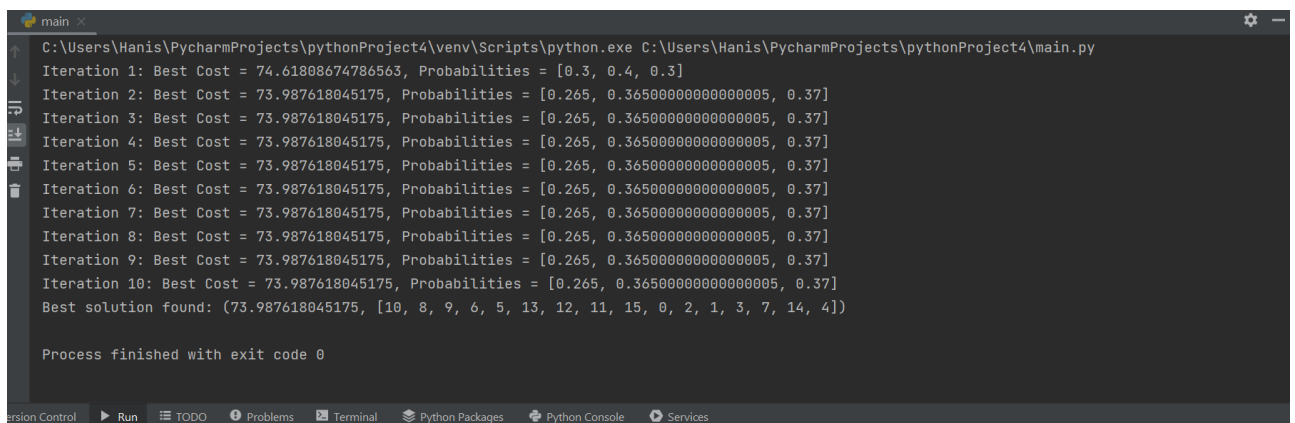
For question 3, we run our codes algorithms on 3 different instances.

The above 2 diagrams show the output from running our codes on Ex5.txt twice.

For the first diagram, From all 10 iterations, it can be seen that the best cost remains 21.82763083055062, whereas the probabilities remain at 0.333333 for all three local search algorithms. With no improvement in the iterations, hence probabilities were not updated. The best solution is found with the same cost along with the tour (3,2,1,0,4)

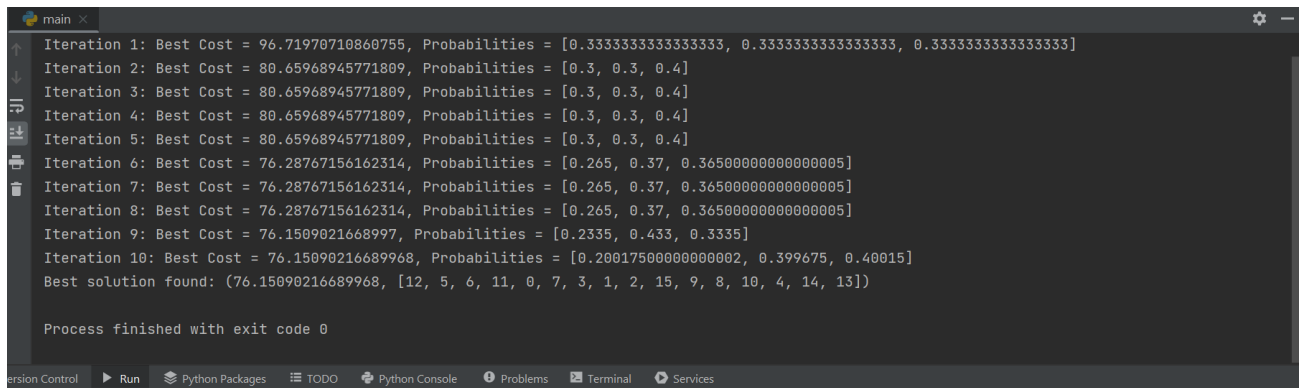
For the second diagram, it can be seen that the cost in first iteration 23.179251 is higher than subsequent iterations, and there is changes in probabilities which suggest that second local search algorithm is able to improve the solution in first iteration, hence allowing for probability change. The best solution is found with same cost with tour (0,1,2,3,4)

Second Experimentation (on Uly16.txt)



```
main
C:\Users\Hanis\PycharmProjects\pythonProject4\venv\Scripts\python.exe C:\Users\Hanis\PycharmProjects\pythonProject4\main.py
Iteration 1: Best Cost = 74.61808674786563, Probabilities = [0.3, 0.4, 0.3]
Iteration 2: Best Cost = 73.987618045175, Probabilities = [0.265, 0.36500000000000005, 0.37]
Iteration 3: Best Cost = 73.987618045175, Probabilities = [0.265, 0.36500000000000005, 0.37]
Iteration 4: Best Cost = 73.987618045175, Probabilities = [0.265, 0.36500000000000005, 0.37]
Iteration 5: Best Cost = 73.987618045175, Probabilities = [0.265, 0.36500000000000005, 0.37]
Iteration 6: Best Cost = 73.987618045175, Probabilities = [0.265, 0.36500000000000005, 0.37]
Iteration 7: Best Cost = 73.987618045175, Probabilities = [0.265, 0.36500000000000005, 0.37]
Iteration 8: Best Cost = 73.987618045175, Probabilities = [0.265, 0.36500000000000005, 0.37]
Iteration 9: Best Cost = 73.987618045175, Probabilities = [0.265, 0.36500000000000005, 0.37]
Iteration 10: Best Cost = 73.987618045175, Probabilities = [0.265, 0.36500000000000005, 0.37]
Best solution found: (73.987618045175, [10, 8, 9, 6, 5, 13, 12, 11, 15, 0, 2, 1, 3, 7, 14, 4])

Process finished with exit code 0
```



```
main ×
Iteration 1: Best Cost = 96.71970710860755, Probabilities = [0.3333333333333333, 0.3333333333333333, 0.3333333333333333]
Iteration 2: Best Cost = 80.65968945771809, Probabilities = [0.3, 0.3, 0.4]
Iteration 3: Best Cost = 80.65968945771809, Probabilities = [0.3, 0.3, 0.4]
Iteration 4: Best Cost = 80.65968945771809, Probabilities = [0.3, 0.3, 0.4]
Iteration 5: Best Cost = 80.65968945771809, Probabilities = [0.3, 0.3, 0.4]
Iteration 6: Best Cost = 76.28767156162314, Probabilities = [0.265, 0.37, 0.36500000000000005]
Iteration 7: Best Cost = 76.28767156162314, Probabilities = [0.265, 0.37, 0.36500000000000005]
Iteration 8: Best Cost = 76.28767156162314, Probabilities = [0.265, 0.37, 0.36500000000000005]
Iteration 9: Best Cost = 76.1509021668997, Probabilities = [0.2335, 0.433, 0.3335]
Iteration 10: Best Cost = 76.15090216689968, Probabilities = [0.20017500000000002, 0.399675, 0.40015]
Best solution found: (76.15090216689968, [12, 5, 6, 11, 0, 7, 3, 1, 2, 15, 9, 8, 10, 4, 14, 13])

Process finished with exit code 0
version Control Run Python Packages TODO Python Console Problems Terminal Services
```

The second experimentation we did was on the Uly16.txt file. We ran our codes algorithms twice too. From the first diagram, it can be seen that the best cost initially is at 74.61, then slowly decreased to 73.98 and decreases over the iterations which indicates the algorithm found better solution as it progresses as the total distance is shorter. The second time of running the code on the same file also suggests that the probabilities adjust over the iterations which indicates different algorithms are improving the solution. The order of the cities visited is also listed in the array at the end.

Third Experimentation (on Uly22.txt)

```
main x
C:\Users\Hanis\PycharmProjects\pythonProject4\venv\Scripts\python.exe C:\Users\Hanis\PycharmProjects\pythonProject4\main.py
Iteration 1: Best Cost = 83.1418218286665, Probabilities = [0.3, 0.3, 0.4]
Iteration 2: Best Cost = 79.15743294085914, Probabilities = [0.265, 0.37, 0.3650000000000005]
Iteration 3: Best Cost = 79.15743294085914, Probabilities = [0.265, 0.37, 0.3650000000000005]
Iteration 4: Best Cost = 76.70437222425751, Probabilities = [0.23325, 0.33825, 0.4285000000000005]
Iteration 5: Best Cost = 76.46796708973673, Probabilities = [0.20016250000000002, 0.4044250000000003, 0.3954125000000006]
Iteration 6: Best Cost = 76.46796708973673, Probabilities = [0.20016250000000002, 0.4044250000000003, 0.3954125000000006]
Iteration 7: Best Cost = 76.46796708973673, Probabilities = [0.20016250000000002, 0.4044250000000003, 0.3954125000000006]
Iteration 8: Best Cost = 76.46796708973673, Probabilities = [0.20016250000000002, 0.4044250000000003, 0.3954125000000006]
Iteration 9: Best Cost = 76.46796708973673, Probabilities = [0.20016250000000002, 0.4044250000000003, 0.3954125000000006]
Iteration 10: Best Cost = 76.46796708973673, Probabilities = [0.20016250000000002, 0.4044250000000003, 0.3954125000000006]
Best solution found: (76.46796708973673, [16, 3, 17, 21, 0, 15, 11, 12, 13, 5, 6, 19, 20, 18, 9, 8, 10, 4, 14, 7, 2, 1])

Process finished with exit code 0
```

```
main x
C:\Users\Hanis\PycharmProjects\pythonProject4\venv\Scripts\python.exe C:\Users\Hanis\PycharmProjects\pythonProject4\main.py
Iteration 1: Best Cost = 89.6853830515861, Probabilities = [0.3, 0.3, 0.4]
Iteration 2: Best Cost = 89.6853830515861, Probabilities = [0.3, 0.3, 0.4]
Iteration 3: Best Cost = 81.47790698781925, Probabilities = [0.265, 0.37, 0.3650000000000005]
Iteration 4: Best Cost = 79.00320774514242, Probabilities = [0.23325, 0.33825, 0.4285000000000005]
Iteration 5: Best Cost = 79.00320774514242, Probabilities = [0.23325, 0.33825, 0.4285000000000005]
Iteration 6: Best Cost = 79.00320774514242, Probabilities = [0.23325, 0.33825, 0.4285000000000005]
Iteration 7: Best Cost = 79.00320774514242, Probabilities = [0.23325, 0.33825, 0.4285000000000005]
Iteration 8: Best Cost = 79.00320774514242, Probabilities = [0.23325, 0.33825, 0.4285000000000005]
Iteration 9: Best Cost = 79.00320774514242, Probabilities = [0.23325, 0.33825, 0.4285000000000005]
Iteration 10: Best Cost = 79.00320774514242, Probabilities = [0.23325, 0.33825, 0.4285000000000005]
Best solution found: (79.00320774514242, [5, 8, 10, 4, 14, 13, 12, 11, 18, 9, 19, 20, 0, 7, 21, 17, 3, 16, 1, 2, 15, 6])

Process finished with exit code 0
```

Question 4

This question works similarly to question 3, however this uses the simulated annealing heuristic, so the variable network search includes additional elements for this. this includes the addition of the initial temperature (T_0), cooling rate (α), and the acceptance probability for non-improving solutions ($\text{math.exp}(-\text{delta} / \text{temperature})$), which are all updated after each iteration- this is in line with the SA concept of occasionally accepting worse solutions in order to escape local optima.

Appendix

Code of algorithm for Question 1

```
import sys
import datetime
import math
import numpy as np
import random

def read_tsp_file(file_name):
    file1 = open(file_name, 'r') # open the file to [r]ead
    lines = file1.readlines() # add file line by line to lines list
    file1.close() # close the file

    location_count = int(lines[0])
    counter = 1
    locations = []

    while counter < len(lines) and lines[counter] is not None:
        current_line = lines[counter]
        parts = current_line.split(",")
        coordinate_x = float(parts[1])
        coordinate_y = float(parts[2])
        new_tuple = (coordinate_x, coordinate_y)
        locations.append(new_tuple)
        counter += 1

    return locations

def calculate_distance(loc1, loc2):
    return math.sqrt(math.pow(loc1[0] - loc2[0], 2) + math.pow(loc1[1] - loc2[1], 2))

def create_distance_matrix(locations):
    table = np.zeros((len(locations), len(locations)))
    for i in range(0, len(locations)):
        for j in range(0, len(locations)):
            table[i, j] = calculate_distance(locations[i], locations[j])
    return table

def move_one_city_to_best_location(tsp_solution, distance_matrix):
    city_list_current = tsp_solution[1]
    cost_current = tsp_solution[0]

    list_length = len(city_list_current) # find the number of locations
    selected_city_location = random.randrange(0, list_length) # randomly select a location
    selected_city = tsp_solution[1][selected_city_location] # find the city that will be moved

    cost_dif = 0 # initialize

    city_before = city_list_current[selected_city_location-1] # even negative python handles
    city_after = city_list_current[(selected_city_location + 1) % list_length] # new index within range
    cost_dif -= distance_matrix[city_before, selected_city] # city_before -> selected_city
    cost_dif -= distance_matrix[selected_city, city_after] # selected_city -> city_after
    cost_dif += distance_matrix[city_before, city_after] # city_before -> city_after

    city_list_out = tsp_solution[1].copy() # city list to return
    city_list_out.remove(selected_city) # remove randomly selected city from this list

    best_location = -1
    best_cost_dif = sys.maxsize
    for i in range(0, list_length-1): # iterate over all possible locations
        city_before2 = city_list_out[i - 1] # if the city is inserted to location i, city before
        city_after2 = city_list_out[i] # if the city is inserted to location i, city after
```

```

current_cost_dif = 0 # this variable shows how much cost difference does inserting to current location makes
current_cost_dif += distance_matrix[city_before2, selected_city] # city_before2 -> selected_city
current_cost_dif += distance_matrix[selected_city, city_after2] # selected_city -> city_after2
current_cost_dif -= distance_matrix[city_before2, city_after2] # city_before2 -> city_after2

if current_cost_dif < best_cost_dif: # if current is better (smaller), update
    best_cost_dif = current_cost_dif
    best_location = i

cost_out = cost_current + cost_dif + best_cost_dif # use the best solution to update
city_list_out.insert(best_location, selected_city) # use the best location to update

return cost_out, city_list_out

def nearest_neighbour_algorithm(distance_matrix):
    max_distance = np.max(distance_matrix)
    list_length = len(distance_matrix)
    city_list = list(range(0, list_length))
    city_list_out = []
    total_cost_out = 0
    modified_distance_matrix = np.copy(distance_matrix)

    first_city = random.randrange(0, list_length)
    city_list.remove(first_city)
    city_list_out.append(first_city)
    current_city = first_city
    modified_distance_matrix[:, current_city] = 2 * max_distance * np.ones(list_length)

    while len(city_list) != 0:
        nearest_next_city = np.argmin(modified_distance_matrix[current_city, :])
        total_cost_out += distance_matrix[current_city, nearest_next_city]
        city_list.remove(nearest_next_city)
        city_list_out.append(nearest_next_city)
        current_city = nearest_next_city
        modified_distance_matrix[:, current_city] = 2 * max_distance * np.ones(list_length)

    total_cost_out += distance_matrix[current_city, first_city]

    return total_cost_out, city_list_out

def solve_tsp2(file_name, max_iter, max_iter2):
    random.seed(5)

    city_locations = read_tsp_file(file_name)
    distance_table = create_distance_matrix(city_locations)

    file1 = open("output.txt", 'w')

    previous_solution = nearest_neighbour_algorithm(distance_table)
    best_solution = previous_solution

    for i in range(0, max_iter):
        current_solution = nearest_neighbour_algorithm(distance_table)

        if current_solution[0] < best_solution[0]:
            best_solution = current_solution

        percentage_change = ((current_solution[0] - previous_solution[0]) / previous_solution[0]) * 100
        if percentage_change < 0:
            percentage_change = 0

        file1.write(f"Iteration {i+1}: Cost = {current_solution[0]}, Improvement = {percentage_change:.5f}%\n")

        previous_solution = current_solution

        start_time = datetime.datetime.now()

        for j in range(0, max_iter2):
            new_solution = move_one_city_to_best_location(current_solution, distance_table)

            if new_solution[0] < current_solution[0]:
                current_solution = new_solution
            if new_solution[0] < best_solution[0]:
                best_solution = new_solution

            file1.write(f"{new_solution[0]} {best_solution[0]}\n")

        end_time = datetime.datetime.now()

        running_time = (end_time - start_time).total_seconds()
        print(f"Iteration {i + 1}: Cost = {current_solution[0]}, "
              f"Improvement = {percentage_change:.5f}%, "
              f"Running Time = {running_time:.7f} seconds")

    file1.close()
    return best_solution

solve_tsp2("ex5.txt", 10, 50)

```

Code of algorithm for Question 2

Code of algorithm for Question 3

```
import sys
import math
import numpy as np
import random

def read_tsp_file(file_name):
    file1 = open(file_name, 'r')
    lines = file1.readlines()
    file1.close()

    location_count = int(lines[0])
    counter = 1
    locations = []

    while counter < len(lines) and lines[counter] is not None:
        current_line = lines[counter]
        parts = current_line.split(",")
        coordinate_x = float(parts[1])
        coordinate_y = float(parts[2])
        new_tuple = (coordinate_x, coordinate_y)
        locations.append(new_tuple)
        counter += 1

    return locations

def calculate_distance(loc1, loc2):
    return math.sqrt(math.pow(loc1[0] - loc2[0], 2) + math.pow(loc1[1] - loc2[1], 2))

def create_distance_matrix(locations):
    table = np.zeros((len(locations), len(locations)))
    for i in range(0, len(locations)):
        for j in range(0, len(locations)):
            table[i, j] = calculate_distance(locations[i], locations[j])
    return table

def nearest_neighbour_algorithm(distance_matrix):
    max_distance = np.max(distance_matrix)
    list_length = len(distance_matrix)
    city_list = list(range(0, list_length))
    city_list_out = []
    total_cost_out = 0
    modified_distance_matrix = np.copy(distance_matrix)
    first_city = random.randrange(0, list_length)
    city_list.remove(first_city)
    city_list_out.append(first_city)
    current_city = first_city
    modified_distance_matrix[:, current_city] = 2 * max_distance * np.ones(list_length)

    while len(city_list) != 0:
        nearest_next_city = np.argmin(modified_distance_matrix[current_city, :])
        total_cost_out += distance_matrix[current_city, nearest_next_city]
        city_list.remove(nearest_next_city)
        city_list_out.append(nearest_next_city)
        current_city = nearest_next_city
```

```

        modified_distance_matrix[:, current_city] = 2 * max_distance * np.ones(list_length)

    total_cost_out += distance_matrix[current_city, first_city]
    return total_cost_out, city_list_out

# Local search algorithm 1: 2-opt
def local_search1(tsp_solution, distance_matrix):
    def local_search1(tsp_solution, distance_matrix):
        best_cost = tsp_solution[0]
        best_route = tsp_solution[1].copy()
        improved = False

        for i in range(1, len(best_route) - 2):
            for j in range(i + 1, len(best_route)):
                new_route = best_route[:]
                new_route[i:j] = best_route[j - 1:i - 1:-1]
                new_cost = sum(distance_matrix[new_route[k]][new_route[k + 1]] for k in range(len(new_route) - 1))
                new_cost += distance_matrix[new_route[-1]][new_route[0]]

                if new_cost < best_cost:
                    best_cost = new_cost
                    best_route = new_route
                    improved = True
                    break
            if improved:
                break

        return best_cost, best_route

# Local search algorithm 2: Swap cities
def local_search2(tsp_solution, distance_matrix):
    best_cost = tsp_solution[0]
    best_route = tsp_solution[1].copy()

    for i in range(len(best_route) - 1):
        for j in range(i + 1, len(best_route)):
            new_route = best_route.copy()
            new_route[i], new_route[j] = new_route[j], new_route[i]
            new_cost = sum(distance_matrix[new_route[k]][new_route[k + 1]] for k in range(len(new_route) - 1))
            new_cost += distance_matrix[new_route[-1]][new_route[0]]

            if new_cost < best_cost:
                best_cost = new_cost
                best_route = new_route

    return best_cost, best_route

# Local search algorithm 3: Simple city move
def local_search3(tsp_solution, distance_matrix):
    best_cost = tsp_solution[0]
    best_route = tsp_solution[1].copy()

    for i in range(len(best_route)):
        for j in range(len(best_route)):

```

```

        if i != j:
            new_route = best_route.copy()
            city = new_route.pop(i)
            new_route.insert(j, city)
            new_cost = sum(distance_matrix[new_route[k]][new_route[k + 1]] for k in range(len(new_route) - 1))
            new_cost += distance_matrix[new_route[-1]][new_route[0]]

            if new_cost < best_cost:
                best_cost = new_cost
                best_route = new_route

    return best_cost, best_route

# Variable Neighbourhood Search
def variable_neighbourhood_search(file_name, max_iter, max_iter2):
    city_locations = read_tsp_file(file_name)
    distance_table = create_distance_matrix(city_locations)
    best_solution = nearest_neighbour_algorithm(distance_table)

    # Initialize the probabilities for selecting each local search algorithm
    probabilities = [1 / 3, 1 / 3, 1 / 3]
    local_search_functions = [local_search1, local_search2, local_search3]

    for i in range(max_iter):
        rnd = random.random()
        cumulative_probability = 0
        for index, prob in enumerate(probabilities):
            cumulative_probability += prob
            if rnd < cumulative_probability:
                current_solution = local_search_functions[index](best_solution, distance_table)
                if current_solution and current_solution[0] < best_solution[0]:
                    best_solution = current_solution
                    # Update probabilities
                    improvement = 0.1 * (1 - probabilities[index])
                    probabilities = [(p - improvement / 2 if i != index else p + improvement) for i, p in
                                     enumerate(probabilities)]
                    # Normalize the probabilities to sum to 1
                    total = sum(probabilities)
                    probabilities = [p / total for p in probabilities]
                    break # Break as we have selected an algorithm

    print(f"iteration {i + 1}: Best Cost = {best_solution[0]}, Probabilities = {probabilities}")

    return best_solution

# Run VNS for TSP
best_tsp_solution = variable_neighbourhood_search("ex5.txt", 10, 50)
print(f"Best solution found: {best_tsp_solution}")

```

Code of algorithm for Question 4

```
import sys
import math
import numpy as np
import random

def read_tsp_file(file_name):
    file1 = open(file_name, 'r')
    lines = file1.readlines()
    file1.close()

    location_count = int(lines[0])
    counter = 1
    locations = []

    while counter < len(lines) and lines[counter] is not None:
        current_line = lines[counter]
        parts = current_line.split(",")
        coordinate_x = float(parts[1])
        coordinate_y = float(parts[2])
        new_tuple = (coordinate_x, coordinate_y)
        locations.append(new_tuple)
        counter += 1

    return locations

def calculate_distance(loc1, loc2):
    return math.sqrt(math.pow(loc1[0] - loc2[0], 2) + math.pow(loc1[1] - loc2[1], 2))

def create_distance_matrix(locations):
    table = np.zeros((len(locations), len(locations)))
    for i in range(0, len(locations)):
        for j in range(0, len(locations)):
            table[i, j] = calculate_distance(locations[i], locations[j])
    return table

def nearest_neighbour_algorithm(distance_matrix):
    max_distance = np.max(distance_matrix)
    list_length = len(distance_matrix)
    city_list = list(range(0, list_length))
    city_list_out = []
    total_cost_out = 0
    modified_distance_matrix = np.copy(distance_matrix)
    first_city = random.randrange(0, list_length)
    city_list.remove(first_city)
    city_list_out.append(first_city)
    current_city = first_city
    modified_distance_matrix[:, current_city] = 2 * max_distance * np.ones(list_length)

    while len(city_list) != 0:
```

```

    nearest_next_city = np.argmin(modified_distance_matrix[current_city, :])
    total_cost_out += distance_matrix[current_city, nearest_next_city]
    city_list.remove(nearest_next_city)
    city_list_out.append(nearest_next_city)
    current_city = nearest_next_city
    modified_distance_matrix[:, current_city] = 2 * max_distance * np.ones(list_length)

total_cost_out += distance_matrix[current_city, first_city]
return total_cost_out, city_list_out

# Local search algorithm 1: 2-opt
def local_search1(tsp_solution, distance_matrix):
    def local_search1(tsp_solution, distance_matrix):
        best_cost = tsp_solution[0]
        best_route = tsp_solution[1].copy()
        improved = False

        for i in range(1, len(best_route) - 2):
            for j in range(i + 1, len(best_route)):
                new_route = best_route[:]
                new_route[i:j] = best_route[j - 1:i - 1:-1]
                new_cost = sum(distance_matrix[new_route[k]][new_route[k + 1]] for k in
range(len(new_route) - 1))
                new_cost += distance_matrix[new_route[-1]][new_route[0]]

                if new_cost < best_cost:
                    best_cost = new_cost
                    best_route = new_route
                    improved = True
                    break
            if improved:
                break

        return best_cost, best_route

# Local search algorithm 2: Swap cities
def local_search2(tsp_solution, distance_matrix):
    best_cost = tsp_solution[0]
    best_route = tsp_solution[1].copy()

    for i in range(len(best_route) - 1):
        for j in range(i + 1, len(best_route)):
            new_route = best_route.copy()
            new_route[i], new_route[j] = new_route[j], new_route[i]
            new_cost = sum(distance_matrix[new_route[k]][new_route[k + 1]] for k in
range(len(new_route) - 1))
            new_cost += distance_matrix[new_route[-1]][new_route[0]]

```



```

        if new_cost < best_cost:
            best_cost = new_cost
            best_route = new_route

    return best_cost, best_route

# Local search algorithm 3: Simple city move
def local_search3(tsp_solution, distance_matrix):
    best_cost = tsp_solution[0]
    best_route = tsp_solution[1].copy()

    for i in range(len(best_route)):
        for j in range(len(best_route)):
            if i != j:
                new_route = best_route.copy()
                city = new_route.pop(i)
                new_route.insert(j, city)
                new_cost = sum(distance_matrix[new_route[k]][new_route[k + 1]] for k in
range(len(new_route) - 1))
                new_cost += distance_matrix[new_route[-1]][new_route[0]]

                if new_cost < best_cost:
                    best_cost = new_cost
                    best_route = new_route

    return best_cost, best_route

def simulated_annealing_vns(file_name, max_iter, max_iter2, T0, alpha):
    city_locations = read_tsp_file(file_name)
    distance_table = create_distance_matrix(city_locations)
    best_solution = nearest_neighbour_algorithm(distance_table)
    current_solution = best_solution
    temperature = T0

    # Initialize the probabilities for selecting each local search algorithm
    probabilities = [1 / 3, 1 / 3, 1 / 3]
    local_search_functions = [local_search1, local_search2, local_search3]

    for i in range(max_iter):
        rnd = random.random()
        cumulative_probability = 0
        for index, prob in enumerate(probabilities):
            cumulative_probability += prob
            if rnd < cumulative_probability:
                new_solution = local_search_functions[index](current_solution, distance_table)

        if new_solution:

```